2 עקרונות ארכיטקטוניים לסוכן

2.1 מבוא

פרק זה מציג את העקרונות הארכיטקטוניים שעל הסטודנטים ליישם בסוכן השחקן והשופט שלהם. העקרונות מבוססים על אותן תבניות שמנהל הליגה משתמש בהן (ראו נספח ח'), אך מותאמים לצרכים הייחודיים של סוכן בודד.

הבדל מהותי

מנהל הליגה מנהל 30 קבוצות ו-60 משחקים במקביל. הסוכן שלכם מנהל שחקן **אחד** או שופט **אחד**. זה משנה את סדרי הגודל אך לא את העקרונות.

2.2 תבנית ה-Orchestrator (שער הכניסה)

2.2.1 הרעיון המרכזי

ה-Orchestrator הוא הרכיב המרכזי שמתאם את כל פעילות הסוכן. הוא משמש כ**שער כניסה יחיד** לכל האינטראקציות עם העולם החיצון.

טבלה 17: תפקידי ה-Orchestrator

תפקיד	תיאור	דוגמה
אתחול	הקמת כל תתי-המערכות	יצירת חיבור ל-Gmail API
תיאום	ניהול זרימת המידע	העברת הודעה ל-Router
ניטור	מעקב אחר תקינות	בדיקת Watchdog
סיום	כיבוי מסודר	שחרור משאבים

2.2.2 מבנה מומלץ

PlayerOrchestrator — Basic Structure

```
class PlayerOrchestrator:
    """Central coordinator for the player agent."""

    def __init__(self, config: Config):
        self.config = config
        self.state_machine = PlayerStateMachine()
        self.rate_limiter = TokenBucketRateLimiter(
            capacity=5, refill_rate=1.0
        )
        self.message_router = MessageRouter()
        self.gmail_client = None
        self._running = False

    def start(self) -> None:
        """Initialize and start the agent."""
        self._setup_gmail_client()
        self._register_handlers()
        self._running = True
        self._run_main_loop()

    def stop(self) -> None:
        """Graceful shutdown."""
        self._running = False
        self._cleanup_resources()
```

2.2.3 עקרון ההפרדה

ה-Orchestrator לא מבצע לוגיקה עסקית בעצמו — הוא רק **מתאם** בין רכיבים:

- **קבלת הודעה** → העברה ל-MessageRouter
- **שינוי מצב** → העברה ל-StateMachine
- **שליחת הודעה** → העברה דרך RateLimiter

2.3 מכונת מצבים לסוכן

2.3.1 מצבי השחקן

הסוכן עובר בין מצבים מוגדרים היטב. כל מצב קובע אילו פעולות מותרות:

טבלה 18: מצבי סוכן השחקן — רישום והקצאות

מצב	תיאור	אירוע מעבר
INIT_START_STATE	מצב התחלתי	REGISTER_REQUEST
REGISTERING	הרשמה בתהליך	REGISTER_SUCCESS
REGISTERED	רשום לליגה	SEASON_START_RECEIVED
AWAITING_ASSIGNMENT	ממתין להזמנת משחק	GAME_INVITATION_RECEIVED

טבלה 19: מצבי סוכן השחקן — משחק

מצב	תיאור	אירוע מעבר
INVITED	קיבל הזמנה למשחק	GAME_ACCEPTED
IN_MATCH	המשחק התחיל	WARMUP_CALL_RECEIVED
WARMUP	שלב חימום	WARMUP_RESPONSE_SENT
QUESTIONING	יצירת שאלות	QUESTIONS_SUBMITTED
AWAITING_ANSWERS	ממתין לתשובות	ANSWERS_RECEIVED
GUESSING	יצירת ניחוש	GUESS_SUBMITTED
MATCH_COMPLETE	משחק הסתיים	GAME_RESULT_RECEIVED
PAUSED	מערכת מושהית	CONTINUE_RECEIVED
ERROR	שגיאה	RESET או RECOVER

2.3.2 תרשים מעברים

מעברים מבוססי אירועים

התרשים מציג את הזרימה העיקרית עם שמות האירועים (בכחול). מצבים נוספים כוללים: AWAITING_ANSWERS, PAUSED, ו-ERROR. לאחר סיום משחק (MATCH_COMPLETE), הסוכן חוזר ל-REGISTERED ממתין למשחק הבא.

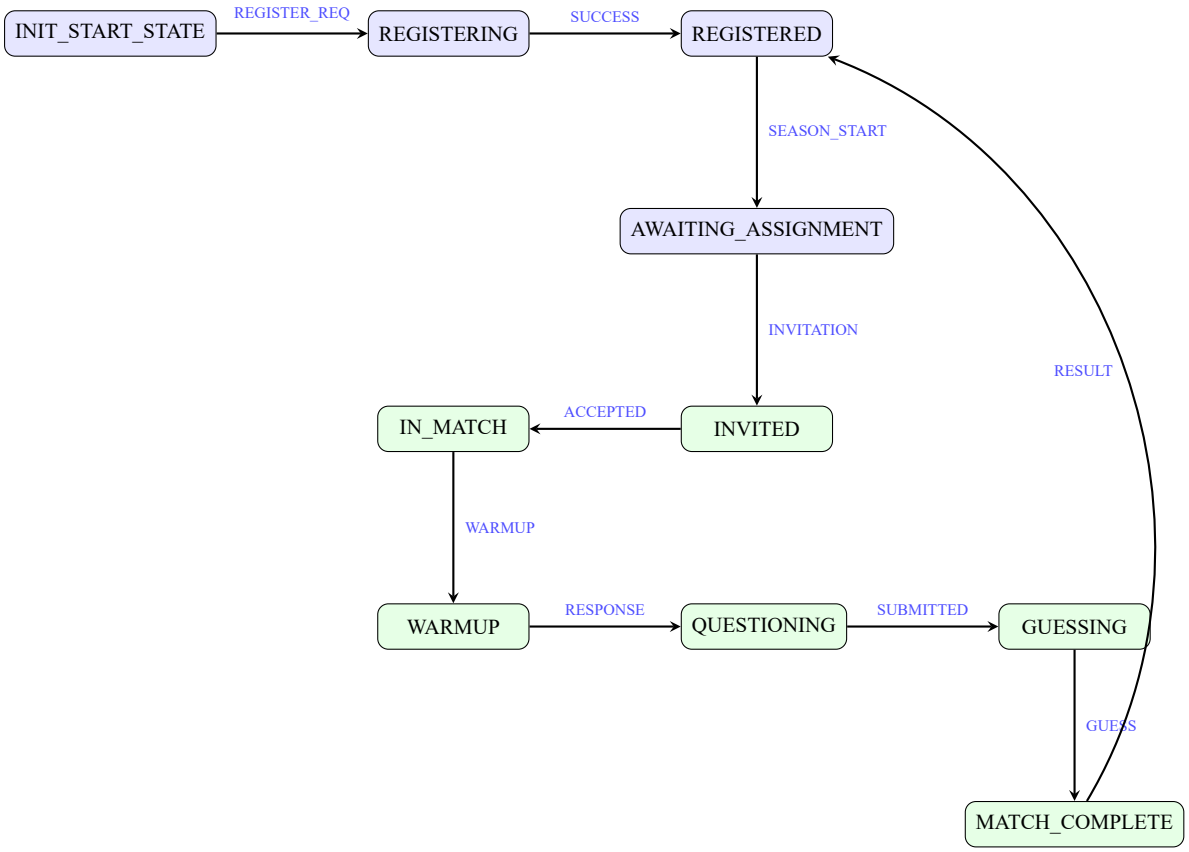
2.3.3 יישום מכונת המצבים

Player State Machine — Event-Based

```
from enum import Enum, auto
from typing import Dict, Tuple, Callable

class PlayerState(Enum):
```


כל הזכויות שמורות - © Dr. Segal Yoram



איור 4: מעברי מצבים מבוססי אירועים בסוכן השחקן

```
    # Registration states
    INIT_START_STATE = auto()
    REGISTERING = auto()
    REGISTERED = auto()
    AWAITING_ASSIGNMENT = auto()
    # Match states
    INVITED = auto()
    IN_MATCH = auto()
    WARMUP = auto()
    QUESTIONING = auto()
    AWAITING_ANSWERS = auto()
    GUESSING = auto()
    MATCH_COMPLETE = auto()
    # System states
    PAUSED = auto()
    ERROR = auto()

class TransitionEvent(Enum):
    REGISTER_REQUEST = auto()
    REGISTER_SUCCESS = auto()
    SEASON_START_RECEIVED = auto()
    GAME_INVITATION_RECEIVED = auto()
    GAME_ACCEPTED = auto()
    WARMUP_CALL_RECEIVED = auto()
    WARMUP_RESPONSE_SENT = auto()
    QUESTIONS_SUBMITTED = auto()
    ANSWERS_RECEIVED = auto()
    GUESS_SUBMITTED = auto()
    GAME_RESULT_RECEIVED = auto()

class PlayerStateMachine:
    # Event-based transitions: (from_state, event) -> to_state
    TRANSITIONS: Dict[Tuple[PlayerState, TransitionEvent], PlayerState]
   = {
        (PlayerState.INIT_START_STATE, TransitionEvent.REGISTER_REQUEST)
   :
            PlayerState.REGISTERING,
        (PlayerState.REGISTERING, TransitionEvent.REGISTER_SUCCESS):
            PlayerState.REGISTERED,
        (PlayerState.REGISTERED, TransitionEvent.SEASON_START_RECEIVED):
            PlayerState.AWAITING_ASSIGNMENT,
        (PlayerState.AWAITING_ASSIGNMENT, TransitionEvent.
   GAME_INVITATION_RECEIVED):
            PlayerState.INVITED,
        (PlayerState.INVITED, TransitionEvent.GAME_ACCEPTED):
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
            PlayerState.IN_MATCH,
        (PlayerState.IN_MATCH, TransitionEvent.WARMUP_CALL_RECEIVED):
            PlayerState.WARMUP,
        (PlayerState.WARMUP, TransitionEvent.WARMUP_RESPONSE_SENT):
            PlayerState.QUESTIONING,
        (PlayerState.QUESTIONING, TransitionEvent.QUESTIONS_SUBMITTED):
            PlayerState.AWAITING_ANSWERS,
        (PlayerState.AWAITING_ANSWERS, TransitionEvent.ANSWERS_RECEIVED)
    :
            PlayerState.GUESSING,
        (PlayerState.GUESSING, TransitionEvent.GUESS_SUBMITTED):
            PlayerState.MATCH_COMPLETE,
        (PlayerState.MATCH_COMPLETE, TransitionEvent.
    GAME_RESULT_RECEIVED):
            PlayerState.REGISTERED,
    }

    def __init__(self):
        self._state = PlayerState.INIT_START_STATE
        self._callbacks: Dict[PlayerState, Callable] = {}

    def transition(self, event: TransitionEvent) -> bool:
        """Attempt event-based state transition."""
        key = (self._state, event)
        if key not in self.TRANSITIONS:
            return False
        self._state = self.TRANSITIONS[key]
        if callback := self._callbacks.get(self._state):
            callback()
        return True

    @property
    def current_state(self) -> PlayerState:
        return self._state
```

מעברים מבוססי אירועים

המימוש האמיתי משתמש במעברים מבוססי **אירועים** (events), לא מעברים ישירים בין מצבים. כל מעבר מופעל על ידי אירוע ספציפי (כגון REGISTER_REQUEST או GAME_ACCEPTED). גישה זו מאפשרת שליטה מדויקת יותר בזרימת המצבים.

2.3.4 מכונת מצבים מתמידה (Persistent State Machine)

עבור סוכן השופט, מומלץ לשמור את מצב מכונת המצבים במסד נתונים כדי לאפשר התאוששות מקריסה. המימוש משתמש בארכיטקטורה מודולרית:

- **Repository Pattern** — הפשטת הגישה למסד הנתונים דרך IStateMachineRepository
- **Mixin Pattern** — PersistentStateMachineMixin מוסיף התמדה לכל מכונת מצבים
- **מעברים מבוססי אירועים** — שימוש ב-transition(event) במקום הגדרת מצב ישירה
- **מתודות מפעל** — restore_from_db() ו-get_or_create() לטעינת מצב

טבלה 20: טבלת state_machine_state

שדה	סוג	תיאור
id	SERIAL PRIMARY KEY	מפתח ראשי אוטומטי
entity_id	VARCHAR(100) NOT NULL	מזהה הישות (למשל ch_id
machine_type	VARCHAR(50) NOT NULL	סוג המכונה: MATCH, ND
current_state	VARCHAR(50) NOT NULL	המצב הנוכחי
state_data	JSONB NOT NULL	נתונים נלווים למצב
parent_id	VARCHAR(100)	מזהה ישות אב (אופציונל
is_terminal	BOOLEAN DEFAULT FALSE	האם מצב סופי
created_at	TIMESTAMP	זמן יצירה
updated_at	TIMESTAMP DEFAULT NOW()	זמן עדכון אחרון
אילוץ: UNIQUE(entity_id, machine_type)		

PersistentStateMachine — Mixin + Repository Pattern

```
# Repository interface for database abstraction
class IStateMachineRepository(ABC):
    @abstractmethod
    def get_state(self, entity_id: str, machine_type: str) -> Optional[
   dict]: ...
    @abstractmethod
    def save_state(self, entity_id: str, machine_type: str,
                   state: str, state_data: dict) -> None: ...

# Mixin adds persistence to any state machine
class PersistentStateMachineMixin:
    """Mixin that adds database persistence to state machines."""

    def __init__(self, repository: IStateMachineRepository,
                 entity_id: str, machine_type: str):
        self._repository = repository
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
        self.entity_id = entity_id
        self.machine_type = machine_type
        # Note: Does NOT auto-load state in constructor

    @classmethod
    def restore_from_db(cls, repository, entity_id: str,
                        machine_type: str) -> Optional['Self']:
        """Factory method to restore state machine from database."""
        row = repository.get_state(entity_id, machine_type)
        if row is None:
            return None  # No default state - returns None if not found
        instance = cls(repository, entity_id, machine_type)
        instance._state = row['current_state']
        instance._state_data = row['state_data']
        return instance

    def transition(self, event: str) -> None:
        """Event-based transition with automatic persistence."""
        old_state = self._state
        super().transition(event)  # Delegate to base state machine
        self._persist_state()

    def _persist_state(self) -> None:
        """Persist current state using repository (upsert pattern)."""
        self._repository.save_state(
            self.entity_id, self.machine_type,
            self._state, self._state_data
        )

# History recording is separate concern
class StateHistoryMixin:
    """Separate mixin for recording state transition history."""
    def record_transition(self, from_state, event, to_state): ...
```

2.3.5 היסטוריית מעברים (State History)

לצורכי ביקורת ודיבאג, מומלץ לתעד כל מעבר מצבים. תיעוד ההיסטוריה ממומש כ-Mixin נפרד (StateHistoryMixin) להפרדת אחריויות:

טבלה 21: טבלת state_history

שדה	סוג	תיאור
entity_id	VARCHAR	מזהה הישות
machine_type	VARCHAR	סוג המכונה
from_state	VARCHAR	מצב מקור
event	VARCHAR	האירוע שגרם למעבר
to_state	VARCHAR	מצב יעד
timestamp	TIMESTAMP	זמן המעבר
is_initial_entry	BOOLEAN	האם כניסה ראשונית

StateHistoryMixin — Separate History Recording

```
class StateHistoryMixin:
    """Mixin for recording state transition history (separate concern)."
    ""

    def __init__(self, history_repository: IStateHistoryRepository, **
    kwargs):
        super().__init__(**kwargs)
        self._history_repo = history_repository

    def record_transition(self, from_state: str, event: str,
                          to_state: str) -> None:
        """Record state transition for audit trail."""
        self._history_repo.insert(
            entity_id=self.entity_id,
            machine_type=self.machine_type,
            from_state=from_state,
            event=event,
            to_state=to_state,
            is_initial_entry=(from_state is None)
        )

# Combining persistence and history via multiple inheritance
class FullStateMachine(PersistentStateMachineMixin,
                       StateHistoryMixin,
                       BaseStateMachine):
    """State machine with both persistence and history recording."""
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
    pass
```

יתרון התמדה

מכונת מצבים מתמידה מאפשרת:

- **התאוששות מקריסה** — הסוכן יכול להמשיך מהמצב האחרון
- **דיבאג** — ניתן לעקוב אחר היסטוריית המעברים
- **ביקורת** — תיעוד מלא לצורכי ערעור

2.4 תבנית ה-GateKeeper (הגנת אימייל)

דרישה קריטית

כל סוכן (שחקן ושופט) חייב לממש מנגנון GateKeeper להגנה על מכסות ה-Gmail API. סוכן ללא הגנה עלול לצרוך את כל המכסה היומית ולהפסיק לתפקד.

2.4.1 שלושת רכיבי ה-GateKeeper

הסוכן שלכם צריך לממש שלושה מנגנוני הגנה:

טבלה 22: רכיבי ה-GateKeeper

רכיב	תפקיד	ערכים מומלצים
Quota Manager	מעקב מכסה יומית	400 הודעות/יום, אזהרה ב-80%
Rate Limiter	מגביל קצב שליחה	5 אסימונים, מילוי 1/שנייה
DOS Detector	זיהוי לולאות אינסופיות	מקסימום 2 הודעות/דקה לאותו נמען

2.4.2 מנהל מכסה (Quota Manager)

Gmail API מגביל חשבונות חינמיים ל-500 הודעות ליום. השאירו מרווח ביטחון:

QuotaManager — Daily Email Limit

```
class QuotaManager:
    """Track daily email quota."""

    def __init__(self, daily_limit: int = 400, warning_threshold: float
   = 0.8):
        self.daily_limit = daily_limit
        self.warning_threshold = warning_threshold
        self._count = 0
```

```
        self._last_reset = datetime.now(timezone.utc).date()

    def can_send(self) -> bool:
        """Check if we can send more emails today."""
        self._maybe_reset()
        return self._count < self.daily_limit

    def record_send(self) -> None:
        """Record an outgoing email."""
        self._maybe_reset()
        self._count += 1
        if self._count >= self.daily_limit * self.warning_threshold:
            logger.warning(f"Quota warning: {self._count}/{self.
   daily_limit}")

    def _maybe_reset(self) -> None:
        """Reset counter at midnight GMT."""
        today = datetime.now(timezone.utc).date()
        if today > self._last_reset:
            self._count = 0
            self._last_reset = today
```

2.4.3 זיהוי לולאות (DOS Self-Protection)

הסוכן עלול להיכנס ללולאה אינסופית של שליחת הודעות. הוסיפו הגנה:

DOS Detector — Prevent Runaway Loops

```
class DOSDetector:
    """Detect runaway message loops."""

    def __init__(self, max_per_minute: int = 2):
        self.max_per_minute = max_per_minute
        self._recent_sends: dict[str, list[datetime]] = {}

    def is_safe_to_send(self, recipient: str) -> bool:
        """Check if sending to this recipient is safe."""
        now = datetime.now(timezone.utc)
        cutoff = now - timedelta(minutes=1)

        # Clean old entries
        if recipient in self._recent_sends:
            self._recent_sends[recipient] = [
                t for t in self._recent_sends[recipient] if t > cutoff
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
            ]

        # Check rate
        recent = self._recent_sends.get(recipient, [])
        return len(recent) < self.max_per_minute

    def record_send(self, recipient: str) -> None:
        """Record a send to recipient."""
        if recipient not in self._recent_sends:
            self._recent_sends[recipient] = []
        self._recent_sends[recipient].append(datetime.now(timezone.utc))
```

למה זה חשוב?

באג בקוד שלכם יכול לגרום לשליחת מאות הודעות בדקות ספורות. זה יגרום:

- מיצוי מכסת האימייל היומית
- חסימה זמנית על ידי Gmail
- הפסדים טכניים בכל המשחקים שנותרו

2.5 מגביל קצב (Token Bucket Rate Limiter)

2.5.1 הבעיה

Gmail API מגביל את מספר הקריאות. חריגה גורמת לשגיאת 429 Too Many Requests וחסימה זמנית.

2.5.2 הפתרון

אלגוריתם Token Bucket מאפשר שליטה חלקה בקצב:

1. **דלי אסימונים** — מכיל עד N אסימונים (למשל, 5)
2. **מילוי** — אסימון חדש כל T שניות (למשל, כל שנייה)
3. **צריכה** — כל קריאת API צורכת אסימון אחד
4. **המתנה** — אם אין אסימונים, ממתינים למילוי

TokenBucketRateLimiter Implementation

```
import time
import threading

class TokenBucketRateLimiter:
```

```python
    """Thread-safe token bucket rate limiter."""

    def __init__(self, capacity: int = 5, refill_rate: float = 1.0):
        self.capacity = capacity
        self.tokens = capacity
        self.refill_rate = refill_rate  # tokens per second
        self.last_refill = time.monotonic()
        self._lock = threading.Lock()

    def acquire(self, timeout: float = 30.0) -> bool:
        """Acquire a token. Blocks until available or timeout."""
        deadline = time.monotonic() + timeout

        while time.monotonic() < deadline:
            with self._lock:
                self._refill()
                if self.tokens >= 1:
                    self.tokens -= 1
                    return True
            time.sleep(0.1)

        return False

    def _refill(self) -> None:
        """Add tokens based on elapsed time."""
        now = time.monotonic()
        elapsed = now - self.last_refill
        new_tokens = elapsed * self.refill_rate
        self.tokens = min(self.capacity, self.tokens + new_tokens)
        self.last_refill = now
```


© Dr. Segal Yoram - כל הזכויות שמורות

המלצה

התחילו עם קיבולת של 5 אסימונים וקצב מילוי של אסימון לשנייה. התאימו לפי הצורך.

2.6 מערכת תזמון (Periodic Handler)

2.6.1 שני לולאות עיקריות

הסוכן צריך לבצע פעולות תקופתיות:

טבלה 23: לולאות תזמון

לולאה	תדירות	תפקיד
Poll Loop	כל 30 שניות	בדיקת הודעות חדשות
Heartbeat Loop	כל 5 דקות	שליחת סימן חיים

2.6.2 יישום

PeriodicHandler with Event

```
import threading
from typing import Callable

class PeriodicHandler:
    """Manages periodic tasks with graceful shutdown."""

    def __init__(self):
        self._stop_event = threading.Event()
        self._threads: list[threading.Thread] = []

    def add_task(self, func: Callable, interval: float, name: str):
        """Add a periodic task."""
        def runner():
            while not self._stop_event.wait(interval):
                try:
                    func()
                except Exception as e:
                    print(f"Error in {name}: {e}")

        thread = threading.Thread(target=runner, name=name, daemon=True)
        self._threads.append(thread)

    def start(self):
```

```
        """Start all periodic tasks."""
        for thread in self._threads:
            thread.start()

    def stop(self, timeout: float = 5.0):
        """Signal all tasks to stop and wait."""
        self._stop_event.set()
        for thread in self._threads:
            thread.join(timeout=timeout)
```

מפרט פרוטוקול

עיקרון חשוב: השתמשו ב-Event.wait() ולא ב-time.sleep() — זה מאפשר כיבוי מהיר ומסודר.

2.7 ניתוב הודעות (Message Router)

2.7.1 תבנית הרישום

במקום if-else ארוך לטיפול בסוגי הודעות, השתמשו בתבנית רישום:

MessageRouter with Handler Registration

```
from typing import Dict, Callable, Any
from enum import Enum

class MessageType(Enum):
    REGISTRATION_CONFIRMED = "REGISTRATION_CONFIRMED"
    MATCH_ASSIGNMENT = "MATCH_ASSIGNMENT"
    GAME_INVITATION = "GAME_INVITATION"
    GAME_JOIN_ACK = "GAME_JOIN_ACK"
    Q21_WARMUP_CALL = "Q21_WARMUP_CALL"
    GAME_RESULT = "GAME_RESULT"
    BROADCAST_KEEP_ALIVE = "BROADCAST_KEEP_ALIVE"

class MessageRouter:
    """Routes messages to registered handlers."""

    def __init__(self):
        self._handlers: Dict[MessageType, Callable] = {}

    def register(self, msg_type: MessageType, handler: Callable):
        """Register a handler for a message type."""
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
        self._handlers[msg_type] = handler

    def route(self, message: Dict[str, Any]) -> bool:
        """Route message to appropriate handler."""
        msg_type_str = message.get("message_type")
        try:
            msg_type = MessageType(msg_type_str)
        except ValueError:
            return False  # Unknown message type

        handler = self._handlers.get(msg_type)
        if handler:
            handler(message)
            return True
        return False
```

2.7.2 יתרונות התבנית

- **הרחבה קלה** — הוספת סוג הודעה חדש = רישום מטפל חדש
- **בדיקות** — קל לבדוק כל מטפל בנפרד
- **קריאות** — ברור מי מטפל במה

2.7.3 מטפלים ייעודיים למשחק Q21 (שופט)

עבור סוכן השופט, יש להגדיר מטפלים ייעודיים לכל שלב במשחק:

טבלה 24: מטפלי Q21 לשופט

מטפל	סוג הודעה	תפקיד
MatchAssignmentHandler	MATCH_ASSIGNMENT	קבלת הקצאת משחק
Q21WarmupResponseHandler	Q21_WARMUP_RESPONSE	אישור מוכנות שחקן
Q21QuestionsBatchHandler	Q21_QUESTIONS_BATCH	קבלת 20 שאלות
Q21GuessSubmissionHandler	Q21_GUESS_SUBMISSION	קבלת ניחוש סופי
ScoreCalculator	(פנימי)	חישוב ציונים
BroadcastHandler	BROADCAST_*	טיפול בשידורים

Q21 Referee Handler Registration

```
class Q21RefereeOrchestrator:
    """Orchestrator for Q21 referee agent."""
```

```
    def _register_handlers(self) -> None:
        """Register all Q21-specific handlers."""
        # Match lifecycle - warmup response confirms player readiness
        self.router.register("MATCH_ASSIGNMENT",
            MatchAssignmentHandler(self.state_machine, self.db))

        # Q21 game flow - warmup serves as readiness confirmation
        self.router.register("Q21_WARMUP_RESPONSE",
            Q21WarmupResponseHandler(self.state_machine))
        self.router.register("Q21_QUESTIONS_BATCH",
            Q21QuestionsBatchHandler(self.state_machine, self.
question_repo))
        self.router.register("Q21_GUESS_SUBMISSION",
            Q21GuessSubmissionHandler(self.state_machine, self.scorer))

        # Broadcasts
        for broadcast_type in BROADCAST_TYPES:
            self.router.register(broadcast_type,
                BroadcastHandler(self.state_machine, self.db))

    def handle_incoming(self, message: dict) -> None:
        """Route incoming message to appropriate handler."""
        msg_type = message.get("message_type")
        handler = self.router.get(msg_type)
        if handler:
            handler.handle(message)
        else:
            self.logger.warning(f"Unknown message type: {msg_type}")
```

עיקרון חשוב

כל מטפל צריך:

- לעדכן את מכונת המצבים בהתאם
- לבטל/להתחיל זמני תפוגה רלוונטיים
- לתעד את הפעולה במסד הנתונים
- להחזיר תגובה אם נדרשת

2.8 איחוד הודעות באמצעות CC

© Dr. Segal Yoram - כל הזכויות שמורות

2.8.1 העיקרון

במקום לשלוח מספר הודעות נפרדות, השתמשו בשדה CC לאיחוד:

טבלה 25: שימוש בשדות נמען

שדה	נמען	תפקיד
TO	מנהל הליגה	הנמען העיקרי
CC	שרת הלוג	תיעוד אוטומטי
CC	חברי קבוצה	שקיפות

2.8.2 דוגמה — שליחת מהלך

Sending with CC

```
def send_move(self, game_id: str, move: str) -> None:
    """Send move to LGM with CC to log server and group."""
    message = self._build_move_message(game_id, move)

    recipients = {
        "to": [self.config.lgm_email],
        "cc": [
            self.config.log_server_email,
            *self._get_group_emails()
        ]
    }

    self.gmail_client.send(
        to=recipients["to"],
        cc=recipients["cc"],
        subject=self._build_subject(game_id),
        body=json.dumps(message)
    )
```

חיסכון ב-API

שליחה אחת עם CC צורכת קריאת API **אחת**, במקום שלוש קריאות נפרדות.

2.9 מנגנון Watchdog

2.9.1 הבעיה

לולאת הסריקה עלולה להיתקע (stall) בגלל:

- תקלת רשת
- חריגה לא מטופלת
- deadlock

2.9.2 הפתרון

Watchdog Implementation

```
import time
import threading

class Watchdog:
    """Detects stalled operations."""

    def __init__(self, timeout: float = 120.0, callback: Callable = None
    ):
        self.timeout = timeout
        self.callback = callback
        self._last_ping = time.monotonic()
        self._lock = threading.Lock()
        self._running = True
        self._thread = threading.Thread(target=self._monitor, daemon=
    True)

    def ping(self):
        """Call this regularly to indicate activity."""
        with self._lock:
            self._last_ping = time.monotonic()

    def _monitor(self):
        """Background monitor thread."""
        while self._running:
            time.sleep(10)  # Check every 10 seconds
            with self._lock:
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
                elapsed = time.monotonic() - self._last_ping
            if elapsed > self.timeout and self.callback:
                self.callback()

    def start(self):
        self._thread.start()

    def stop(self):
        self._running = False
```

2.9.3 שימוש

Watchdog in Main Loop

```
def _run_main_loop(self):
    watchdog = Watchdog(
        timeout=120.0,
        callback=self._handle_stall
    )
    watchdog.start()

    while self._running:
        watchdog.ping()  # Signal activity
        messages = self._poll_for_messages()
        for msg in messages:
            self.message_router.route(msg)
        time.sleep(30)
```

2.10 תבנית מעקב מועדים (Deadline Tracker)

2.10.1 הבעיה

אם הסוכן לא מגיב בזמן, התגובה תידחה והוא יקבל הודעת REJECTION_NOTIFICATION.
עליכם לעקוב אחר מועדי התגובה.

DeadlineTracker — Track Response Deadlines

```
from dataclasses import dataclass
from datetime import datetime, timezone

@dataclass
class PendingResponse:
```

```
    message_id: str
    message_type: str
    deadline: datetime
    correlation_id: str

class DeadlineTracker:
    """Track pending responses and their deadlines."""

    def __init__(self):
        self._pending: dict[str, PendingResponse] = {}

    def add_pending(self, message_id: str, message_type: str,
                    deadline: datetime, correlation_id: str) -> None:
        """Track a message requiring response."""
        self._pending[message_id] = PendingResponse(
            message_id=message_id,
            message_type=message_type,
            deadline=deadline,
            correlation_id=correlation_id
        )

    def get_urgent(self) -> list[PendingResponse]:
        """Get messages with deadline in next 30 seconds."""
        now = datetime.now(timezone.utc)
        cutoff = now + timedelta(seconds=30)
        return [p for p in self._pending.values() if p.deadline <=
   cutoff]

    def mark_responded(self, correlation_id: str) -> None:
        """Remove message after responding."""
        to_remove = [k for k, v in self._pending.items()
                     if v.correlation_id == correlation_id]
        for k in to_remove:
            del self._pending[k]
```


© Dr. Segal Yoram - כל הזכויות שמורות

2.10.2 מנהל זמני תפוגה מתמיד (Persistent Timeout Manager)

עבור סוכן השופט, מומלץ לנהל זמני תפוגה במסד נתונים לאפשר התאוששות מקריסה:

טבלה 26: טבלת pending_timeouts

שדה	סוג	תיאור
id	SERIAL PRIMARY KEY	מפתח ראשי אוטומטי
timeout_id	VARCHAR(100) UNIQUE NOT NULL	מזהה ייחודי (UUID)
timeout_type	VARCHAR(50) NOT NULL	סוג: WARMUP, QUESTIONS, GUESS
target_email	VARCHAR(255) NOT NULL	כתובת השחקן הממתין
started_at	TIMESTAMP NOT NULL	זמן התחלת הספירה
timeout_seconds	FLOAT NOT NULL	משך הזמן בשניות (תומך בשברים)
expires_at	TIMESTAMP NOT NULL	זמן תפוגה (מחושב)
context	JSONB	הקשר נוסף (match_id, שלב, ניסיון)
created_at	TIMESTAMP	זמן יצירת הרשומה

הערות מימוש

- **match_id** נשמר בתוך שדה ה-context ולא כעמודה נפרדת
- **ביטול timeout** מתבצע על ידי **מחיקת** הרשומה (לא סימון is_active=FALSE)
- timeout_id הוא UUID ולא מספר סידורי

טבלה 27: ערכי זמני תפוגה במימוש השופט

שלב	זמן (שניות)	הערות
WARMUP	300	ניסיון חוזר אחד מותר
QUESTIONS	600	ללא ניסיון חוזר
GUESS	600	ללא ניסיון חוזר
ברירת מחדל	900	לפעולות כלליות

PersistentTimeoutManager — Database-Backed Timeouts

```
class PersistentTimeoutManager:
    """Manages game timeouts with database persistence."""

    TIMEOUT_VALUES = {
        "WARMUP": 300.0,     # 5 minutes (float)
        "QUESTIONS": 600.0, # 10 minutes
        "GUESS": 300.0,      # 5 minutes
        "DEFAULT": 900.0,    # 15 minutes
```

```
    }

    def __init__(self, db_connection):
        self.db = db_connection

    def start_timeout(self, timeout_type: str, target_email: str,
                      match_id: str = None, context: dict = None) -> str
:
        """Start a new timeout and persist to database."""
        timeout_id = str(uuid.uuid4())  # UUID string, not SERIAL
        timeout_seconds = self.TIMEOUT_VALUES.get(timeout_type,
                                                  self.TIMEOUT_VALUES["
DEFAULT"])
        # match_id stored inside context JSONB, not separate column
        full_context = {"match_id": match_id, **(context or {})}
        self.db.execute("""
            INSERT INTO pending_timeouts
                (timeout_id, timeout_type, target_email, started_at,
                 timeout_seconds, expires_at, context)
            VALUES (%s, %s, %s, NOW(), %s,
                    NOW() + INTERVAL '%s seconds', %s)
        """, (timeout_id, timeout_type, target_email, timeout_seconds,
              timeout_seconds, json.dumps(full_context)))
        return timeout_id  # Returns UUID string

    def cancel_timeout(self, timeout_id: str) -> None:
        """Cancel timeout by DELETING (not marking inactive)."""
        self.db.execute("""
            DELETE FROM pending_timeouts WHERE timeout_id = %s
        """, (timeout_id,))

    def get_expired_timeouts(self) -> list[dict]:
        """Get all expired timeouts (existence = active)."""
        return self.db.execute("""
            SELECT * FROM pending_timeouts WHERE expires_at <= NOW()
        """).fetchall()
```

2.11 תבנית התאמת הודעות (Correlation Matching)

2.11.1 שימוש ב-correlation_id

כל הודעה במערכת כוללת שדה correlation_id המקשר בין בקשה לתגובה:

© Dr. Segal Yoram - כל הזכויות שמורות

MessageCorrelator — Match Requests and Responses

```
class MessageCorrelator:
    """Match outgoing requests with incoming responses."""

    def __init__(self):
        self._open_requests: dict[str, dict] = {}

    def track_request(self, correlation_id: str, request: dict) -> None:
        """Track an outgoing request."""
        self._open_requests[correlation_id] = {
            "request": request,
            "sent_at": datetime.now(timezone.utc),
            "status": "OPEN"
        }

    def match_response(self, correlation_id: str, response: dict) ->
   dict:
        """Match response to original request."""
        if correlation_id not in self._open_requests:
            return None

        entry = self._open_requests[correlation_id]
        entry["response"] = response
        entry["received_at"] = datetime.now(timezone.utc)
        entry["status"] = "CLOSED"
        return entry["request"]

    def get_open_requests(self) -> list[str]:
        """Get all open correlation IDs."""
        return [k for k, v in self._open_requests.items()
                if v["status"] == "OPEN"]
```

2.12 טיפול בשידורים (Broadcast Handling)

2.12.1 11 סוגי שידורים

מנהל הליגה שולח הודעות שידור לכל המשתתפים. חלקן דורשות תגובה:

טבלה 28: סוגי שידורים ותגובה נדרשת

סוג	תיאור	תגובה?	מועד
BROADCAST_KEEP_ALIVE	בדיקת זמינות	כן	30s
BROADCAST_CRITICAL_RESET	איפוס מערכת	כן	2m
BROADCAST_CRITICAL_PAUSE	השהיית פעילות	כן	2m
BROADCAST_CRITICAL_CONTINUE	המשך פעילות	כן	2m
BROADCAST_FREE_TEXT	הודעה חופשית	אופציונלי	1h
BROADCAST_NEW_LEAGUE_ROUND	תחילת מחזור	לא	—
BROADCAST_END_LEAGUE_ROUND	סיום מחזור	לא	—
BROADCAST_START_SEASON	תחילת עונה	לא	—
BROADCAST_END_SEASON	סיום עונה	לא	—
BROADCAST_ASSIGNMENT_TABLE	טבלת הקצאות	לא	—
BROADCAST_ROUND_RESULTS	תוצאות מחזור	לא	—

2.12.2 יישום מטפל שידורים

BroadcastHandler — Complete Implementation

```
class BroadcastHandler:
    """Handles league-wide broadcast messages."""

    # Broadcasts that require response
    RESPONSE_REQUIRED = {
        "BROADCAST_KEEP_ALIVE": ("RESPONSE_KEEP_ALIVE", 30),
        "BROADCAST_CRITICAL_RESET": ("RESPONSE_CRITICAL_RESET", 120),
        "BROADCAST_CRITICAL_PAUSE": ("RESPONSE_CRITICAL_PAUSE", 120),
        "BROADCAST_CRITICAL_CONTINUE": ("RESPONSE_CRITICAL_CONTINUE",
   120),
    }

    def __init__(self, state_machine: PlayerStateMachine):
        self.state_machine = state_machine
        self._previous_state = None

    def handle(self, message: dict) -> dict | None:
        """Route broadcast to appropriate handler."""
        msg_type = message.get("message_type")
```


כל הזכויות שמורות - © Dr. Segal Yoram

```
        if msg_type == "BROADCAST_KEEP_ALIVE":
            return self._handle_keep_alive(message)
        elif msg_type == "BROADCAST_CRITICAL_PAUSE":
            return self._handle_pause(message)
        elif msg_type == "BROADCAST_CRITICAL_CONTINUE":
            return self._handle_continue(message)
        elif msg_type in ("BROADCAST_NEW_LEAGUE_ROUND",
                          "BROADCAST_END_LEAGUE_ROUND"):
            self._update_round_state(message)
            return None  # No response required
        return None

    def _handle_keep_alive(self, message: dict) -> dict:
        """Respond to keep-alive with current state."""
        return {
            "message_type": "RESPONSE_KEEP_ALIVE",
            "broadcast_id": message.get("broadcast_id"),
            "machine_state": self.state_machine.current_state.name
        }
```

זיהוי שידור

ניתן לזהות הודעת שידור לפי:

- סוג ההודעה מתחיל ב-_BROADCAST
- קיים שדה broadcast_id בתוכן ההודעה
- נשלחה מכתובת מנהל הליגה

2.12.3 תיעוד שידורים במסד נתונים (שופט)

עבור סוכן השופט, מומלץ לתעד שידורים שהתקבלו ותגובות שנשלחו:

טבלה 29: טבלת broadcasts_received

שדה	סוג	תיאור
id	SERIAL PRIMARY KEY	מפתח ראשי אוטומטי
broadcast_id	VARCHAR(100) UNIQUE NOT NULL	מזהה השידור
message_type	VARCHAR(100) NOT NULL	סוג השידור
message_text	VARCHAR(200)	תוכן טקסט קצר
league_id	VARCHAR(100)	מזהה הליגה (אופציונלי)
round_id	VARCHAR(100)	מזהה המחזור (אופציונלי)
season_id	VARCHAR(100)	מזהה העונה (אופציונלי)
sender_email	VARCHAR(255) NOT NULL	כתובת השולח
payload	JSONB NOT NULL	תוכן השידור
requires_response	BOOLEAN DEFAULT FALSE	האם נדרשת תגובה
response_deadline	TIMESTAMP	מועד אחרון לתגובה (אופציונלי)
response_sent	BOOLEAN DEFAULT FALSE	האם נשלחה תגובה
response_sent_at	TIMESTAMP	מתי נשלחה התגובה
received_at	TIMESTAMP DEFAULT NOW()	זמן קבלה
processed_at	TIMESTAMP	מתי עובד השידור
status	VARCHAR(50) DEFAULT 'RECEIVED'	סטטוס: RECEIVED, RESPONDED, RED

כל הזכויות שמורות - © Dr. Segal Yoram

טבלה 30: טבלת broadcast_responses_sent

שדה	סוג	תיאור
broadcast_id	VARCHAR (מפתח זר)	קישור לשידור המקורי
response_message_id	VARCHAR(100)	מזהה הודעת התגובה
response_type	VARCHAR	סוג התגובה
payload	JSONB	תוכן התגובה
sent_at	TIMESTAMP	זמן שליחה
status	VARCHAR	סטטוס: SENT, FAILED
context_season_id	VARCHAR	הקשר עונה
context_round_id	VARCHAR	הקשר מחזור

Persistent Broadcast Tracking

```
class PersistentBroadcastHandler:
    """Tracks broadcasts in database for audit and recovery."""

    def __init__(self, db_connection, state_machine):
        self.db = db_connection
        self.state_machine = state_machine

    def record_broadcast(self, broadcast: dict) -> None:
        """Record incoming broadcast to database."""
        msg_type = broadcast.get("message_type")
        requires_response = msg_type in self.RESPONSE_REQUIRED

        self.db.execute("""
            INSERT INTO broadcasts_received
                (broadcast_id, message_type, payload, requires_response,
                 response_deadline, received_at, status)
            VALUES (%s, %s, %s, %s, %s, NOW(), 'PENDING')
            ON CONFLICT (broadcast_id) DO NOTHING
        """, (broadcast.get("broadcast_id"), msg_type,
              json.dumps(broadcast), requires_response,
              self._calculate_deadline(msg_type)))

    def record_response(self, broadcast_id: str, response: dict) -> None
    :
        """Record outgoing response to database."""
        self.db.execute("""
            INSERT INTO broadcast_responses_sent
                (broadcast_id, response_message_id, response_type,
    payload, sent_at)
```

```
        VALUES (%s, %s, %s, %s, NOW())
    """, (broadcast_id, response.get("message_id"),
          response.get("message_type"), json.dumps(response)))

    # Mark broadcast as responded
    self.db.execute("""
        UPDATE broadcasts_received
        SET response_sent = TRUE, status = 'RESPONDED'
        WHERE broadcast_id = %s
    """, (broadcast_id,))
```

2.13 מדיניות אזורי זמן

2.13.1 עיקרון מנחה

כלל ברזל

כל השעונים הפנימיים במערכת פועלים ב-UTC. התצוגה למשתמש תהיה בשעון ישראל חורף (UTC+2).

עיקרון זה מבטיח:

- **עקביות** — כל הסוכנים משתמשים באותו בסיס זמן
- **פשטות** — אין צורך להתמודד עם שעון קיץ/חורף בלוגיקה הפנימית
- **תיעוד** — כל הלוגים והמסדי נתונים משתמשים ב-UTC
- **התאמה** — המרה לזמן מקומי מתבצעת רק בשכבת התצוגה

2.13.2 שעון ישראל חורף

שעון ישראל חורף הוא UTC+2 (שעתיים לפני UTC). לדוגמה:

טבלה 31: המרת זמנים

UTC	שעון ישראל חורף	הערה
00:00	02:00	חצות ב-UTC = 02:00 בישראל
10:00	12:00	בוקר ב-UTC = צהריים בישראל
22:00	00:00 (+1)	ערב ב-UTC = חצות למחרת בישראל

כל הזכויות שמורות - © Dr. Segal Yoram

2.13.3 יישום בקוד

Timezone Handling — UTC Internal

```
from datetime import datetime, timezone, timedelta
from zoneinfo import ZoneInfo  # Python 3.9+

# Israel Winter Time = UTC+2
ISRAEL_WINTER_TZ = ZoneInfo("Asia/Jerusalem")

def get_current_utc() -> datetime:
    """Get current time in UTC (for internal use)."""
    return datetime.now(timezone.utc)

def utc_to_israel_display(utc_time: datetime) -> str:
    """Convert UTC to Israel time for display."""
    israel_time = utc_time.astimezone(ISRAEL_WINTER_TZ)
    return israel_time.strftime("%d/%m/%Y %H:%M:%S")

def format_deadline_for_user(deadline_utc: datetime) -> str:
    """Format deadline for user-facing messages."""
    israel_time = deadline_utc.astimezone(ISRAEL_WINTER_TZ)
    return f"ורחא דעומ: {israel_time.strftime('%H:%M:%S')} ווע( לארשי)
   "

# Example usage
deadline_utc = datetime(2026, 1, 15, 10, 30, 0, tzinfo=timezone.utc)
print(utc_to_israel_display(deadline_utc))  # "15/01/2026 12:30:00"
```

2.13.4 נקודות חשובות

1. **אחסון במסד נתונים** — כל חותמות הזמן נשמרות ב-UTC
2. **לוגים** — כל הלוגים משתמשים ב-UTC לצורך עקביות
3. **פרוטוקול התקשורת** — כל הזמנים בהודעות JSON הם ב-UTC
4. **תצוגה למשתמש** — המרה ל-UTC+2 רק בממשק המשתמש
5. **איפוס מכסות** — מכסת ה-Gmail API מתאפסת בחצות UTC (כלומר 02:00 בישראל)

למה לא שעון קיץ?

הליגה פועלת בחודשי החורף (ינואר-פברואר), לכן משתמשים בשעון ישראל חורף (UTC+2). אם הליגה תתקיים בקיץ, יש להשתמש ב-UTC+3.

2.14 ניהול תצורה

2.14.1 הפרדת סודות מהגדרות

טבלה 32: הפרדת קבצי תצורה

קובץ	תוכן	ב-Git?
.env	סודות (סיסמאות, מפתחות)	**לא**
config.json	הגדרות (כתובות, פרמטרים)	כן

2.14.2 קובץ סודות

.env File

```
# Gmail API credentials
GMAIL_CLIENT_ID=your-client-id.apps.googleusercontent.com
GMAIL_CLIENT_SECRET=your-client-secret
GMAIL_REFRESH_TOKEN=your-refresh-token

# Database
DATABASE_URL=postgresql://user:pass@localhost/league
```

2.14.3 קובץ הגדרות

config.json File

```
{
  "agent": {
    "player_id": "P-Q21G-001",
    "group_id": "G001"
  },
  "gmail": {
    "poll_interval_seconds": 30,
    "rate_limit_capacity": 5,
    "rate_limit_refill_rate": 1.0
  },
```


כל הזכויות שמורות - © Dr. Segal Yoram

```
  "lgm": {
    "email": "league.manager.q21g@gmail.com",
    "log_server_email": "beit.halevi.700@gmail.com"
  },
  "watchdog": {
    "timeout_seconds": 120
  }
}
```

2.15 סיכום

פרק זה הציג את העקרונות הארכיטקטוניים ליישום סוכן שחקן או שופט:

- **Orchestrator** — שער כניסה יחיד לתיאום כל הפעילות
- **מכונת מצבים** — ניהול מעברים בין מצבים מוגדרים
- **Token Bucket** — שליטה בקצב קריאות API
- **Periodic Handler** — ביצוע פעולות תקופתיות עם כיבוי מסודר
- **Message Router** — ניתוב הודעות למטפלים רשומים
- **איחוד CC** — חיסכון בקריאות API ושקיפות
- **Watchdog** — זיהוי והתאוששות מתקיעות
- **טיפול בשידורים** — תגובה להודעות מערכתיות
- **מדיניות אזורי זמן** — שעונים פנימיים ב-UTC, תצוגה בשעון ישראל (UTC+2)
- **הפרדת תצורה** — סודות ב-.env, הגדרות ב-config.json

מפרט פרוטוקול

המלצה: התחילו ביישום פשוט של כל רכיב, ושפרו בהדרגה. אל תנסו לבנות הכל מושלם מההתחלה.

לרקע על הארכיטקטורה המקבילה של מנהל הליגה, ראו נספח ח'.